

Assignment 2 — Object-Oriented Programming

“CampusWheels” Vehicle-Rental Desk

192-201 · Advanced Computer Programming with Generative AI
Siam University — Faculty of Information Technology (International)

1. Background

Siam University runs a small campus vehicle-rental desk. The staff want a simple program to keep track of the vehicles they rent out and the members who rent them. Some vehicles are ordinary cars and motorbikes, and some are electric cars.

Your job is to model this with Python classes. A vehicle can be rented and returned. A renter's details must always be valid. The different vehicle types should each describe themselves in their own way. In doing this you practise the three core ideas of object-oriented programming: classes and objects, encapsulation (keeping data valid), and inheritance with polymorphism (special types that share a common parent).

2. What to create (and where)

Create the files below and write all the code yourself.

rental.py — put all your classes in this one file

Vehicle

- Data: make, model, plate, and is_rented (starts False).
- rent() sets it to rented; return_vehicle() sets it back to available.
- __str__() prints one clear line, e.g. Toyota Yaris (1AB234) [available].

Renter

- Data: name, license_no, and rented (an empty list).
- name must not be empty, and license_no must be a positive number — otherwise raise ValueError.
- Use @property so the check runs whenever the value is set, and keep the real value private. The same check must also run when the renter is first created.

ElectricCar (inherits from Vehicle)

- Adds one new piece of data: battery_kwh. Calls super().__init__(...). Overrides __str__ so it looks different from a plain vehicle.

Motorbike (inherits from Vehicle)

- Adds one new piece of data: engine_cc. Calls super().__init__(...). Overrides __str__ in its own way.

main.py — a short program that shows everything works

Runs with python main.py and demonstrates:

- create a few vehicles and a renter; rent and return a vehicle; print them;
- try to create a renter with a bad name or bad licence, and show the ValueError is caught;
- put a Vehicle, an ElectricCar and a Motorbike in one list and print each in a loop — each prints in its own format.

README.md — a few lines

- How to run your program, and an honest note on how you used AI (you may ask AI to explain ideas, but you write the code).

Keep these names exactly: Vehicle, Renter, ElectricCar, Motorbike, and the methods rent(), return_vehicle(), __str__ — otherwise the tests cannot find your code.

3. What your program must pass (pytest)

To pass the test, your program must cover the following:

Vehicle & Renter

- A new vehicle starts available; rent() makes it rented; return_vehicle() makes it available again.
- __str__ shows the status, and the line changes after the vehicle is rented.
- A new renter's rented list starts empty.

Encapsulation

- Creating a renter with an empty name raises ValueError.
- Creating a renter with a zero or negative licence raises ValueError.
- The same check also fires if the value is changed later.

Inheritance & polymorphism

- ElectricCar and Motorbike are treated as a Vehicle (isinstance is true).
- They still have the parent's data and can rent() / return_vehicle().
- Each type's __str__ is different, so printing a mixed list gives three different lines.

4. Commit to GitHub

Create a new folder name “Assignment_02” on your GitHub repository and commit these three files:

```
rental.py    # your classes
main.py      # your demonstration program
README.md    # how to run + your AI-use note (optional)
```

Submit the link to your repository. Tip: commit each part as you finish it, so your history shows your progress.